

Gestion de code avec Git (FIEC30EM)

frank.buloup@univ-amu.fr

Aix Marseille Université
Faculté des Sciences du Sport
Master 2 IEAP - Facteurs Humains



Gestion de code avec Git

- 1 Introduction
- 2 Utilisation & Installation de Git
- 3 Projet
- 4 Annexes

1

- Organisation & Objectifs du module
- Le compte GitHub & les projets
- Git ? C'est quoi ?
- Petit historique des systèmes de contrôle de versions

0

- Cours et TDs pour apprendre les bases de Git
- Un projet par équipe pour mettre en œuvre ces apprentissages
- Deux (à trois) étudiants par équipe

- Environ 4 heures de cours/TDs
- Environ 14 heures pour le projet

1000

- 50% sur un CC (examen écrit 30mn)
- 50% sur le projet tout au long du module

À l'issue de cette formation, vous serez capable de :

- 1 Décrire les principes de fonctionnement d'un gestionnaire de versions distribué
- 2 Créer, initialiser et cloner un dépôt avec Git
- 3 Manipuler les commandes de Git pour gérer les fichiers et les branches
- 4 Résoudre les conflits de fusion
- 5 Mettre en œuvre un projet en mode collaboratif avec Git sur GitHub

Comptes GitHub

- Création de comptes sur GitHub
- Compte du Master : MasterFHGit

Les projets

Sur la base d'un Arduino Uno R4 Wifi avec :

- une jauge de contrainte (2 postes)
- un potentiomètre
- une résistance à détection de force (2 postes)
- un capteur à ultrason (2 postes)
- un codeur optique



Une idée ?

Documentation Git

Git n'est pas un serveur (au sens centralisé)

- Git n'offre pas de fonctionnalité de gestion de projet complète (e.g. gestion de tâches, de bugs ou wiki)
- Github et GitLab sont deux exemples de solutions complètes



Github



GitLab



GitLab

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100

-

[illegible]

Git est un système de contrôle de version (Version Control System)

- Historique des versions de tous les fichiers
- Manipulation complète de l'historique possible
- Décentralisé (bientôt plus clair...)
- On trouve aussi : "Source Code Management System"

Git est un système de contrôle de version (Version Control System)

- Historique des versions de tous les fichiers
- Manipulation complète de l'historique possible
- Décentralisé (bientôt plus clair...)
- On trouve aussi : "Source Code Management System"

Pourquoi gérer des versions de fichiers, utiliser un VCS ?

- Pour pouvoir revenir en arrière
- Pour comparer des versions de fichiers
- Pour expérimenter, tester des idées
- Pour travailler à plusieurs plus facilement

Vous pouvez gérer tous types de fichiers avec git !

Exemples de fichiers gérés par git

- Fichiers de code informatique
- Fichiers textes (e.g. .doc)
- Fichiers binaires (e.g. images, sons) - Si volumineux : git LFS*
- Fichiers de données - Si volumineux : git LFS*
- Documentation de projets (cahier des charges, devis...)
- etc.

Gestion collaborative de projets

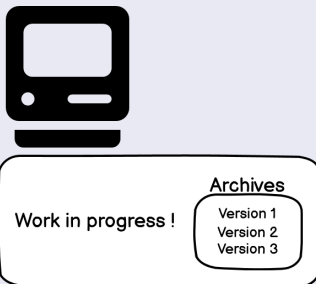
* Utilisation de l'extension **Git Large File Storage**

Taille max. de 5GB sur Github (AWS Amazon S3)

Un peu d'histoire !

Local VCS - Avant... il y a longtemps !

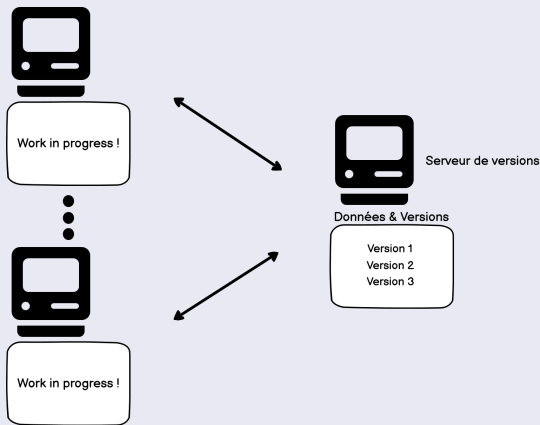
- Système D : horodatage, classement dossiers, compression etc.
- Pro : Revision Control System (1982, projet GNU*)



* GNU : GNU's Not Unix

Centralized VCS - Avant... un peu moins longtemps !

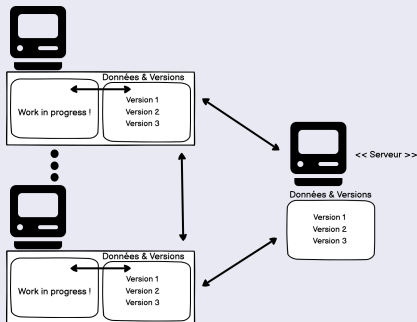
- Partage plus facile
- Connexion réseau obligatoire (approche Client-Serveur)



e.g. Concurrent Version System (CVS), Subversion (SVN)

Distributed VCS - Maintenant, depuis un moment (début 2000) !

- État "serveur" et réseau pas critique (approche Pair-à-Pair)
- On peut remonter un "serveur" à partir des autres machines
- Les opérations locales (la majorité) sont rapides
- Travail privé possible



e.g. Git, Mercurial, Darcs

L'origine de Git

- Créé en 2005 par [Linus TORVALDS](#)
- Pour le noyau Linux
- Activement maintenu par [Junio HAMANO](#)
- Popularisé par GitHub
- On prononce "guit" et pas "jit" !
- Git = Crétin (= Conna... !)

Git - Wikipedia

Les principales caractéristiques de Git

- Contrôle de version distribué (pair-à-pair)
- Gestion des branches de code (création, suppression, fusion)
- Opérations atomiques (Commits)
- Données de gestion stockée dans un répertoire .git

2

- Les différents modes d'utilisation
- Utilisation sur GitHub
- Installation & utilisation en local - Les commandes de base
- Les zones de Git
- Annuler un commit : commande revert
- Modifier l'historique des commits : commande reset
- Fusionner et rebaser des branches - Résolution de conflits
- Collaboration

Comment utiliser Git ?

- En ligne de commande (Terminal & Git Bash pour Windows)
- Outils graphiques intégrés à Git : "git-gui" et "gitk"
- Avec une IHM dans un OS (GitKraken, SourceTree) : [ici](#)
- Intégré dans un EDI (plugin, package etc.)
- Sur internet : github, gitlab, bitbucket etc.



GitKraken



SourceTree

L'apprentissage en ligne de commande est le plus efficace !

-
- The screenshot shows the GitHub Dashboard interface. On the left sidebar, under 'Create your first project', the 'Create repository' button is highlighted with a red circle. The main content area shows the 'Home' page with options to start writing code, create a new repository, and introduce yourself with a profile README. The 'Create a new repository' section is crossed out with a red 'X'. On the right, there are sections for 'Explore repositories' featuring repositories like 'scolladon / sfdx-git-delta' and 'eclipsesource / jsonforms'.

Gestion de code avec Git (FIEC30EM) 18/ 85

Required fields are marked with an asterisk (*).

Owner *

MasterFHGit - / FirstRepository

✓ FirstRepository is available.

Great repository names are short and memorable. Need inspiration? How about [improved-octo-winner](#) ?

Description (optional)

☒ Public

Anyone on the internet can see this repository. You choose who can commit.

☐ Private

You choose who can see and commit to this repository.

Initialize this repository with:

☒ Add a README file

This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: None

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

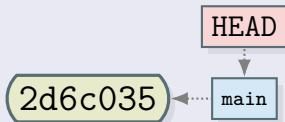
This will set `main` as the default branch. Change the default name in your [settings](#).

① You are creating a public repository in your personal account.

Create repository

Figure: Création du dépôt "FirstRepository" public avec fichier README

État initial après le premier commit sur GitHub



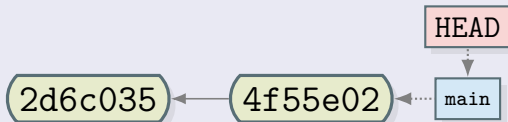
- **2d6c035** est l'ID du commit (l'ID complet est un SHA1)
- **main** est une référence au dernier commit de la branche qui porte le même nom, sur le dépôt distant
- On peut avoir plusieurs branches et donc plusieurs références (**main**, **dev**, **feature1** etc.)
- On trouvera aussi **master** à la place de **main**
- **main** ou **master** sont par convention des noms de branches de production

Exercice I - Premier pas avec GitHub

- ### 3 Modifier le fichier README.md sur GitHub

- _____

Downloaded from <http://ajph.org/> on November 10, 2014



Heads

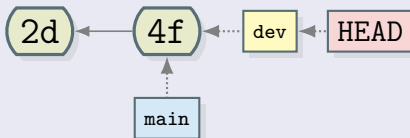
A head is simply a reference to a commit object. Each head has a name. By default, there is a head in every repository called master (or main). A repository can contain any number of heads. At any given time, one head is selected as the “current head.” This head is aliased to HEAD, always in capitals.

Note this difference: a “head” (lowercase) refers to any one of the named heads in the repository; “HEAD” (uppercase) refers exclusively to the currently active head. This distinction is used frequently in Git documentation.

Git - Heads

-

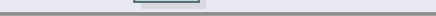
1. *Journal of Management Studies*, 1997, 34, 1, 1-14.



-



0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99



Concepts importants à retenir

- Branches
- Commits
- Heads (références, pointeurs)
- Toute branche a son pointeur de même nom qui référence toujours le dernier commit de la branche
- HEAD est le seul pointeur qui peut bouger. C'est à partir de lui que sera créé le prochain commit.

Exercice II - Installation de Git en local

- 1 Installer Git en suivant le lien suivant : [Git - SCM](#)

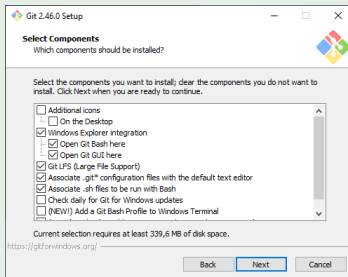
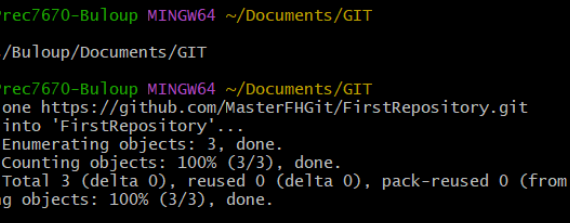


Figure: Git Setup - Garder les options par défaut

- 2 Créer un dossier GIT sur votre machine
- 3 Importer votre dépôt FirstRepository dans ce dossier



The screenshot shows a Windows terminal window with the title bar "MINGW64:/c/Users/Buloup/Documents/GIT/FirstRepository". The terminal content is as follows:

```
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT
$ pwd
/c/Users/Buloup/Documents/GIT

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT
$ git clone https://github.com/MasterFHGit/FirstRepository.git
Cloning into 'FirstRepository'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

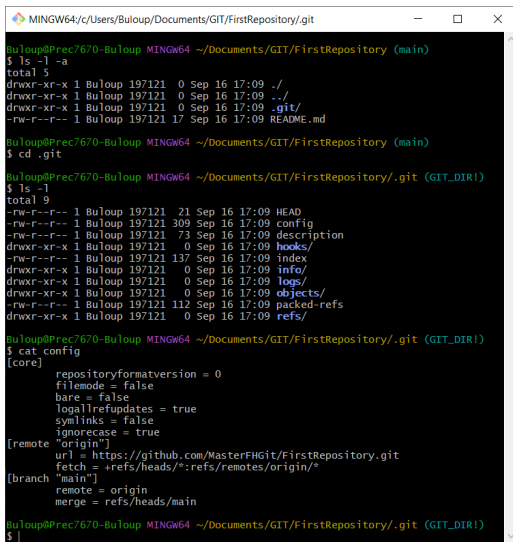
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT
$ git remote -v
fatal: not a git repository (or any of the parent directories): .git

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT
$ cd FirstRepository/

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (main)
$ git remote -v
origin https://github.com/MasterFHGit/FirstRepository.git (fetch)
origin https://github.com/MasterFHGit/FirstRepository.git (push)

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (main)
$ |
```

Figure: Commandes clone et remote



```
MINGW64/c/Users/Buloup/Documents/GIT/FirstRepository/.git

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (main)
$ ls -l -a
total 5
drwxr-xr-x 1 Buloup 197121 0 Sep 16 17:09 ./
drwxr-xr-x 1 Buloup 197121 0 Sep 16 17:09 ../
drwxr-xr-x 1 Buloup 197121 0 Sep 16 17:09 .git/
-rw-r--r-- 1 Buloup 197121 17 Sep 16 17:09 README.md

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (main)
$ cd .git

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository/.git (GIT_DIR!)
$ ls -l
total 9
-rw-r--r-- 1 Buloup 197121 21 Sep 16 17:09 HEAD
-rw-r--r-- 1 Buloup 197121 309 Sep 16 17:09 config
-rw-r--r-- 1 Buloup 197121 73 Sep 16 17:09 description
drwxr-xr-x 1 Buloup 197121 0 Sep 16 17:09 hooks/
-rw-r--r-- 1 Buloup 197121 137 Sep 16 17:09 index
drwxr-xr-x 1 Buloup 197121 0 Sep 16 17:09 info/
drwxr-xr-x 1 Buloup 197121 0 Sep 16 17:09 logs/
drwxr-xr-x 1 Buloup 197121 0 Sep 16 17:09 objects/
-rw-r--r-- 1 Buloup 197121 112 Sep 16 17:09 packed-refs
drwxr-xr-x 1 Buloup 197121 0 Sep 16 17:09 refs/

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository/.git (GIT_DIR!)
$ cat config
[core]
    repositoryformatversion = 0
    filemode = false
    bare = false
    logallrefupdates = true
    symlinks = false
    ignorecase = true
[remote
"origin"]
    url = https://github.com/MasterFHGit/FirstRepository.git
    fetch = +refs/heads/*:refs/remotes/origin/*
[branch
"main"]
    remote = origin
    merge = refs/heads/main

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository/.git (GIT_DIR!)
$ |
```

Figure: Dossier .git et fichier config

remote "origin" ?

- origin est un alias local donné au dépôt distant
- il peut être changé dans le fichier config ...

```
MINGW64:/c:/Users/Buloup/Documents/GIT/FirstRepository/.git
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository/.git (GIT_DIR!)
$ vim config
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository/.git (GIT_DIR!)
$ cat config
[core]
    repositoryformatversion = 0
    filemode = false
    bare = false
    logallrefupdates = true
    symlinks = false
    ignorecase = true
[remote "toto"]
    url = https://github.com/MasterFHGit/FirstRepository.git
    fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
    remote = toto
    merge = refs/heads/main
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository/.git (GIT_DIR!)
$ |
```

Figure: Changer l'alias origin

... ou lors de la commande clone

```
MINGW64:/c/Users/Buloup/Documents/GIT/FirstRepository
$ git clone https://github.com/MasterFHGit/FirstRepository.git -o toto
Cloning into 'FirstRepository'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT
$ cd FirstRepository/

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (main)
$ git remote -v
toto    https://github.com/MasterFHGit/FirstRepository.git (fetch)
toto    https://github.com/MasterFHGit/FirstRepository.git (push)

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (main)
$ cat ./git/config
[core]
    repositoryformatversion = 0
    filemode = false
    bare = false
    logallrefupdates = true
    symlinks = false
    ignorecase = true
[remote "toto"]
    url = https://github.com/MasterFHGit/FirstRepository.git
    fetch = +refs/heads/*:refs/remotes/toto/*
[branch "main"]
    remote = toto
    merge = refs/heads/main

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (main)
$ |
```

Figure: Changer l'alias origin

Petit résumé intermédiaire

Quelques Commandes : clone, status, log, switch

```
1 $ git clone url           # Clone repository
2 $ git remote -v          # Remotes infos
3 $ git status              # Git status
4 $ git log                 # Print commits
5 $ git log --oneline       # Print abbrev-commits
6 $ git switch branchName  # Move HEAD to branchName
```

Il existe un répertoire .git

- Il contient la base de donnée locale du dépôt : dépôt local
- Il contient un fichier de configuration du dépôt

```
MINGW64/c/Users/Buloup/Documents/GIT/FirstRepository
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT
$ mkdir FirstRepository && cd FirstRepository
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository
$ git init
Initialized empty Git repository in C:/Users/Buloup/Documents/GIT/FirstRepository/.git/
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (master)
$ git remote add origin https://github.com/MasterFHGit/FirstRepository.git
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (master)
$ git fetch origin
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (9/9), 2.62 KiB | 38.00 KiB/s, done.
From https://github.com/MasterFHGit/FirstRepository
 * [new branch]      main -> origin/main
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (master)
$ git checkout main
Switched to a new branch 'main'
branch 'main' set up to track 'origin/main'.
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (main)
$ ls
README.md
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (main)
$ cat ./.git/config
[core]
    repositoryformatversion = 0
    filemode = false
    bare = false
    logallrefupdates = true
    symlinks = false
    ignorecase = true
[remote "origin"]
    url = https://github.com/MasterFHGit/FirstRepository.git
    fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
    remote = origin
    merge = refs/heads/main
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (main)
$ |
```

Figure: Importer un dépôt distant - Version 2

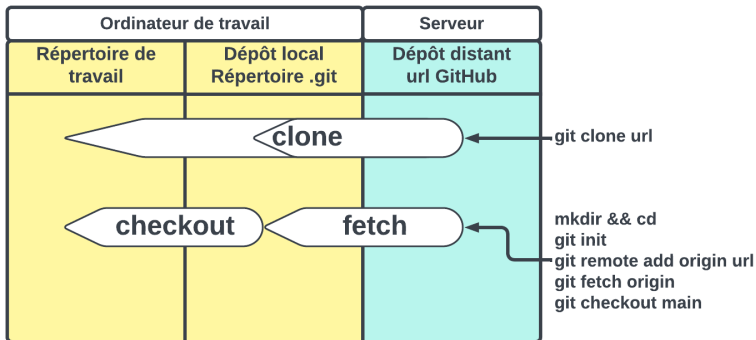


Figure: Importer un dépôt distant - Les deux versions

- 1 Changer le fichier README sur github (branche main)
- 2 Reporter ces changements en local

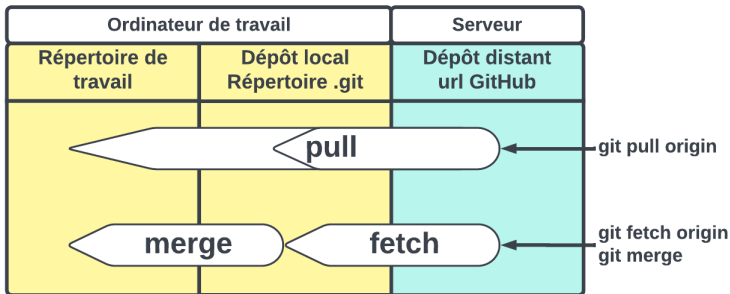


Figure: Mettre à jour un dépôt local - Les deux versions

```

~/Documents/GIT/FirstRepository on main ✓
git pull origin
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 904 bytes | 301.00 KiB/s, done.
From https://github.com/MasterFHGit/FirstRepository
   4c8e792..a6ab892  main       -> origin/main
Updating 4c8e792..a6ab892
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)

~/Documents/GIT/FirstRepository on main ✓

```

Figure: Mettre à jour un dépôt local - Version 1

Commande fetch & option -dry-run

Rien n'est modifié dans le dépôt local

Figure: Le dépôt local est-il à jour ? Non !

```
README.md | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```

Figure: Maintenant oui !

```
1 $ git pull origin # Get from origin
2 $ git fetch origin && git merge # Idem !
3 $ git fetch origin --dry-run # Has origin changed ?
4 $ git log # Print commits
5 $ git log --oneline # Print abbrev-commits
```

Exercice IV - Changements à partir du dépôt local

- 1 Changer le fichier README en local (branche main)
- 2 Reporter ces changements en distant

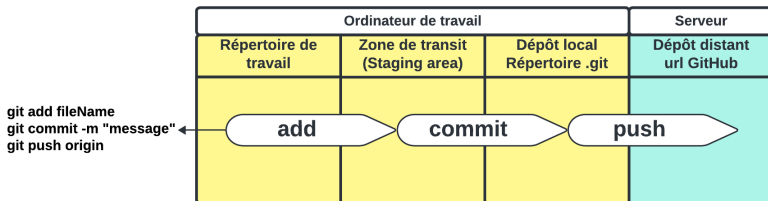


Figure: Mettre à jour le dépôt distant



Figure: Zone de transit

Commandes add, commit & push

```
FirstRepository — frank@mbp-de-frank — ..rstRepository — zsh — 98x37

~/Documents/GIT/FirstRepository on main
~/micro README.md

~/Documents/GIT/FirstRepository on main !1
git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

~/Documents/GIT/FirstRepository on main !1
git add README.md

~/Documents/GIT/FirstRepository on main +1
git commit README.md -m "Changed from local repository"
[main 47b514b] Changed from local repository
1 file changed, 1 insertion(+)

~/Documents/GIT/FirstRepository on main +1
git push origin
Username for 'https://github.com': MasterFHGit
Password for 'https://MasterFHGit@github.com':
remote: Support for password authentication was removed on August 13, 2021.
remote: Please see https://docs.github.com/get-started/getting-started-with-git/about-remote-repositories#cloning-with-https-urls for information on currently recommended modes of authentication.
fatal: Authentication failed for 'https://github.com/MasterFHGit/FirstRepository.git/'

~/Documents/GIT/FirstRepository on main +1 128 x took 24s
```

Figure: Mettre à jour le dépôt distant



Figure: Génération d'un jeton dans GitHub

Génération d'un jeton d'accès personnel

- Conserver ce jeton. Il n'est visible qu'une seule fois !
- C'est bien de lui donner un petit nom (note) !
- Ne pas oublier de sélectionner l'option "repo"
- Sur un dépôt privé, rien n'est possible sans jeton

Pour plus d'infos : [Personal Access Token \(PAT\)](#)



```
FirstRepository — frank@mbp-de-frank — ..rstRepository — -zsh — 80x24
~/Documents/GIT/FirstRepository on main +1 128 x
git push origin
Username for 'https://github.com': MasterFHGit
Password for 'https://MasterFHGit@github.com':
remote: Support for password authentication was removed on August 13, 2021.
remote: Please see https://docs.github.com/get-started/getting-started-with-git/about-remote-repositories#cloning-with-https-urls for information on currently recommended modes of authentication.
fatal: Authentication failed for 'https://github.com/MasterFHGit/FirstRepository.git/'

~/Documents/GIT/FirstRepository on main +1 128 x took 14s
git push origin
Username for 'https://github.com': MasterFHGit
Password for 'https://MasterFHGit@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 293 bytes | 293.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/MasterFHGit/FirstRepository.git
f263e1c..8bdc17a  main -> main

~/Documents/GIT/FirstRepository on main ✓ took 7s
```

Figure: Avec le jeton comme mdp, le push fonctionne

Pour enregistrer le jeton

```
1 $ git config --global credential.helper store
```

... et ne plus avoir à le retaper qu'une seule fois. Le jeton est enregistré dans le fichier texte **.git-credentials** de votre répertoire personnel.

Pour signer les commits automatiquement

```
1 $ git config --global user.email "mail@somewhere.com"
2 $ git config --global user.name "username"
```

Signed-off-by: username <mail@somewhere.com>

Pour cloner (ou autre) on peut aussi faire :

```
1 $ git clone https://uname:pass@github.com/uname/rp.git
```

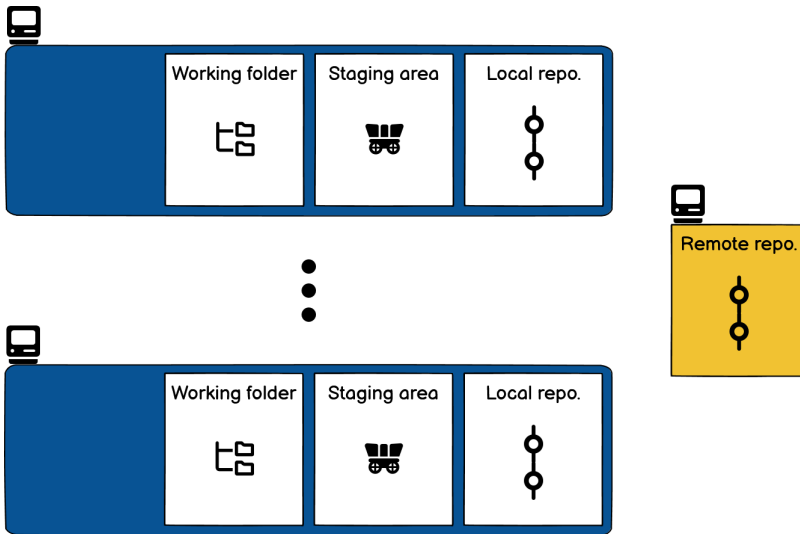


```
1 $ git add fileName # Stage fileName
2 $ git add . # Stage modified and new files
3 $ git add -u # Stage modified and deleted files
4 $ git add -A # Stage everything
5 $ git commit -m "A commit message"
```

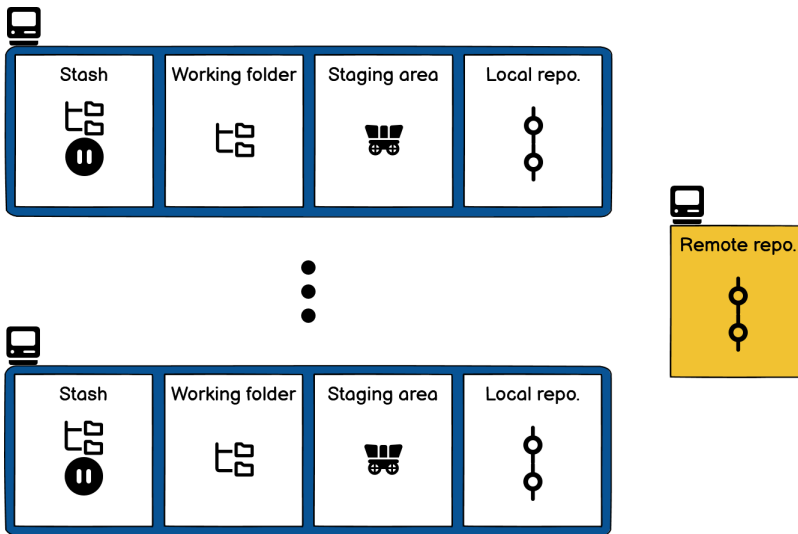
D'autres commandes utiles ! switch, branch, checkout

1	\$	git	switch	branchName	# Switch to branchName
2	\$	git	checkout	52fd6e4	# In detached HEAD
3	\$	git	checkout	branchName	# HEAD on branchName
4	\$	git	branch	-D branchName	# To delete branchName
5	\$	git	branch		# List all local branches
6	\$	git	branch	-r	# List all remote branches
7	\$	git	branch	-a	# List all branches
8	\$	git	branch	newBranchName	# Create new branch
9	\$	git	switch	-c neBrNa	# Create & switch to neBrNa

Les zones de Git



Les zones de Git



stash = réserve, remise

Exercice V - Commande revert

- 1 Ajouter un nouveau commit dans la branche **main** en modifiant le fichier README.md (Before revert)
- 2 Utiliser la commande revert pour annuler ce commit
- 3 Utiliser la commande checkout pour vous déplacer dans les commits

Exercice V - Commande revert

- ➊ Ajouter un nouveau commit dans la branche **main** en modifiant le fichier README.md (Before revert)
- ➋ Utiliser la commande revert pour annuler ce commit
- ➌ Utiliser la commande checkout pour vous déplacer dans les commits

Commande revert

```
1 $ git switch main           # Goto main branch
2 $ vim README.md             # Modify README
3 $ git commit -am "Before revert" # Commit
4 $ git revert HEAD           # Revert last commit
5 $ git log --oneline         # See logs
6 $ cat README.md             # File content
7 $ git checkout 44e5761      # Goto previous commit
8 $ cat README.md             # File content
```

Commande revert

- Elle ne modifie pas l'historique des commits (pas de suppression de commits)
- Elle crée un nouveau commit
- Elle détermine comment inverser les changements introduits par le commit spécifié et ajoute un nouveau commit avec le contenu inversé qui en résulte

Exercice VI - Commande reset

- 1 Modifier le fichier README.md sans faire de commit
- 2 Supprimer les deux derniers commits (le Before revert et le revert) précédents en utilisant la commande reset
- 3 Conclusion

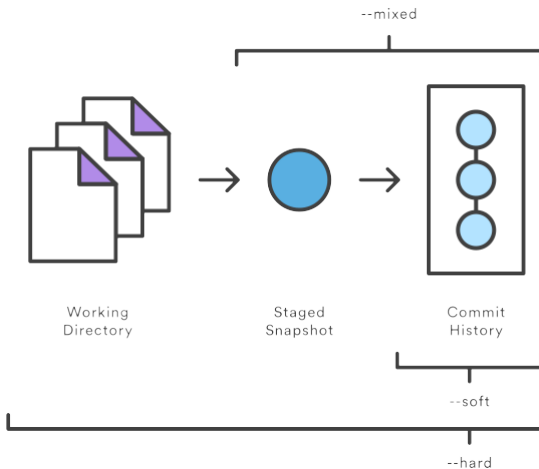


Figure: Commande reset - Les différents modes

1. *Journal of Management Studies*, 1997, 34, 1, 1-14.

- Elle revient dans l'état du commit spécifié en mettant l'index (staging area) à jour mais pas la copie de travail
- Elle modifie l'historique des commits (supprime des commits)
- La commande restore permet de récupérer l'index : `git restore README.md`
- C'est l'option par défaut : mode mixte

"git reset idCommit" et "git reset --mixed idCommit" sont donc des commandes indentiques

Exercice VII - Préparation fusion et rebase

- ## 1 Ajouter un nouveau commit dans la branche **dev**

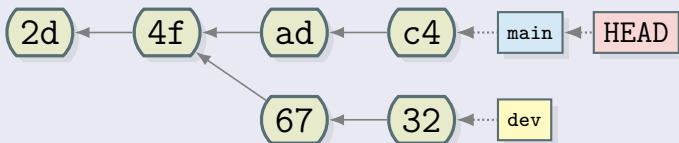
Exercice VII - Préparation fusion et rebase

- ## 1 Ajouter un nouveau commit dans la branche **dev**

Préparation fusion et rebase

```
1 $ git switch dev           # Goto dev branch
2 $ vim README.md           # Modify README
3 $ git commit -am "devC1"  # Commit 1 in dev branch
```

État actuel du dépôt local



Comment récupérer les modifications de **dev** dans **main** ?

- Les branches **dev** et **main** ont divergées
- Problème de fusion de branches : gestion des conflits

Exercice VIII - Fusion de branches avec conflits

- ## 1 Fusionner la branche dev dans la branche main en local

Fusionner : la commande merge

```
1 $ git switch main    # Goto main branch
2 $ git merge dev      # Merge dev branch to main branch
```



```

[ ~/Documents/GIT/FirstRepository on main merge ~1 ✓ ]
git status
On branch main
Your branch is up to date with 'origin/main'.

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

[ ~/Documents/GIT/FirstRepository on main merge ~1 ✓ ]
cat README.md
# FirstRepository

First commit from main branch

<<<<<< HEAD
seconde commit from main branch

third commit from main branch - local
=====
First commit from dev branch
>>>>>> dev

[ ~/Documents/GIT/FirstRepository on main merge ~1 ✓ ]

```

Figure: Résultat de la fusion

Exercice VIII - Fusion de branches avec conflits

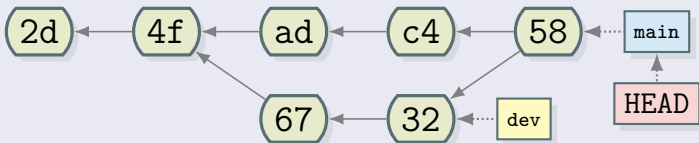
- 1 Fusionner la branche dev dans la branche main en local
- 2 Modifier le fichier README.md
- 3 Ajouter ce fichier au staging (marquer la résolution de conflits)
- 4 Faire le commit

Fusionner : la commande merge

```
1 $ git switch main      # Goto main branch
2 $ git merge dev        # Merge dev branch to main branch
3 $ vim README.md        # Modify README.md
4 $ git add README.md    # To mark resolution
5 $ git commit -m "dev->main_{}_Conflicts_{}_resolution"
```

Ne pas hésiter à utiliser la commande `git status` régulièrement

État du dépôt local après la fusion - Merge



Les commits de la branche *dev* ont été fusionnés avec la branche *main* dans le dernier commit de cette branche (58)

Exercice IX - Rebase de branches avec conflits

- 1 Revenir dans l'état du dépôt distant (supprimer puis cloner de nouveau le dépôt distant)
- 2 Ajouter un nouveau commit dans la branche **dev** en local
- 3 Rebaser la branche dev dans la branche main en local
- 4 Modifier le fichier README.md
- 5 Ajouter ce fichier au staging pour marquer la résolution de conflits
- 6 Terminer le rebase

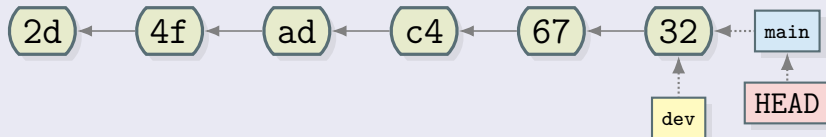
Rebaser : la commande rebase

```

1 $ git switch dev # Goto dev branch
2 $ vim README.md # Edit readme
3 $ git commit -am "devC1" # New commit in dev
4 $ git switch main # Goto main
5 $ git rebase main dev # Rebase dev in main
6 $ vim README.md # Resolve conflicts
7 $ git add README.md # Mark resolved
8 $ git rebase --continue # Continue rebase
9 $ git switch main # Goto main
10 $ git merge dev # Merge dev in main
11 $ git branch -d dev # Delete dev (-D?)

```

État du dépôt local après le rebase



Tous les commits de la branche *dev* ont été mis à la suite de ceux de la branche *main*

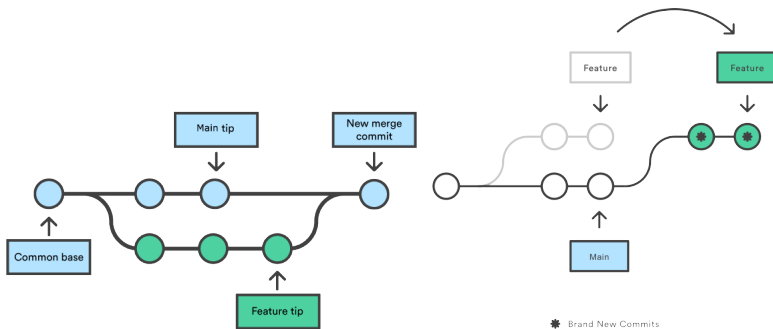


Figure: Illustration des commandes merge à gauche et rebase à droite

Travail en équipe

- Chaque développeur doit avoir un compte GitHub
- Chaque développeur doit avoir un PAT
- Le propriétaire du dépôt origine doit envoyer des invitations

Pour plus d'infos : **Inviter des collaborateurs**

3

[Veuillez suivre ce lien](#)

Pour avoir un superbe terminal !

Suivre ce lien : 👍 [Oh my posh !](#) 👍
Installation sous Windows

Pour avoir un autre superbe terminal !

Suivre ce lien : [Oh my bash !](#)

Installation :

```
1 $ bash -c "$(curl -fsSL https://raw.githubusercontent.com/ohmybash/oh-my-bash/master/tools/install.sh)"
```

Désinstallation :

```
1 $ uninstall_oh_my_bash
```

Modification du thème et des plugins du Terminal alors possible

Encore une alternative : **Oh my zsh !**



Figure: Logiciels SourceTree, GitHub et GitKraken Desktop

Trois outils graphiques libres et très utiles

Contexte

J'ai des fichiers sur mon ordi que je souhaite gérer sur GitHub

Solution

- 1 Créer le dépôt local si pas déjà fait
- 2 Créer le dépôt distant sur GitHub **VIDE** (évite les erreurs) :
 - Pas de fichier README.md, .gitignore, licence etc.
 - Récupérer l'URL du dépôt distant
- 3 Ajouter cet URL en tant que dépôt distant
- 4 Pousser la branche principale sur le dépôt distant

```
1 $ git init
2 $ git add ...
3 $ git commit ...
4 $ git remote add origin REMOTE-URL
5 $ git remote -v # Check URL has been added
6 $ git push -u origin main
```

-479,7 : 7 lignes **à partir de** la 479 dans le commit 7e46066
+484,7 : 7 lignes **à partir de** la 484 dans le commit 3a78b4a


```
1 # Changes between workspace and stage
2 $ git diff
3 # Changes between workspace + stage and last commit
4 $ git diff HEAD
```

```
diff --git a/presentation.pdf b/presentation.pdf
index eb2bfe9..326de97 100644
Binary files a/presentation.pdf and b/presentation.pdf differ
diff --git a/presentation.tex b/presentation.tex
index a02e9fe..4f1635d 100644
--- a/presentation.tex
+++ b/presentation.tex
@@ -1165,36 +1165,6 @@ Pour plus d'infos : \textcolor{blue}{\textbf{\und
\end{frame}

\section{Les projets}

-
-
-\begin{frame}
-\begin{alertblock}{Rien à disposition pour faire des mesures objectives}
-\begin{itemize}
```

Les index présents en entête de la sortie d'un diff sont des identifiants internes à git, pas des numéros de commits !

Comparaison de fichiers entre dépôts distant et local

```
1 # - Workspace and stage diffs are ignored -
2 # Without git fetch, changes between origin and local
3 # repos won't be synchronised and diff will be blind.
4 # Use git branch -a to list local and "remote local"
5 # branches. e.g. : branchName and
6 # remotes/origin/branchName
7 # In order to update "remote local" branches ->
8 $ git fetch
9 # Changes between main branches : local branch main
10 # and "remote local" branch remotes/origin/main ->
11 $ git diff main origin/main
12 # Changes in path/tofile between origin
13 # and local repos in main branches ->
14 $ git diff origin/main:path/tofile main:path/tofile
15 $ ...
16 $ git merge origin/main
```

76/ 85

```
* 2186499 Update reset and go further topics
*   af46e63 Merge conflict
| \
| * 7e46066 Add to go further
| * 654b4e9 Change font size and add git SCM link
* | fd02601 Add clone command with uname and passwd
| /
* db8324e Remove pong image
* 7149293 Update revert and reset commands
* e8896ef Update merge and rebase commands
* 48b507e Global update 3
* 6b16542 Global update 2
* e44f652 Global update
* fe4cd53 Ajout des commandes rebase et merge
* b6f584b Add annexes
*   74a5511 Resolve conflict
| \
| * 8971e4d Some minor changes
* | e24286b Update projects
| /
* de0dca4 Remove spaces before $ in lstlisting env.
* dedb9fb Add credential and Signed-Off alerts blocks
* 8ff1a7f Update projects part
* 73cc805 Update projects and git commands
* f6ae20c Add main concepts slides
* c979f94 Add gitDAG library to draw nice git graph
```

```
1 $git log --graph --decorate --abbrev-commit --all --pretty=oneline
```

Un superbe graphe de commits dans la console 😊!





Git Alias

```
alias gs=' git status '
alias ga=' git add '
alias ga=' git push'
alias c=' git commit '
```

Documentation Alias sur Git

Les alias en pratique

```
1 # Creating an alias to get a
2 # graph of commits in console
3 $ git config --global alias.lg 'log --graph --decorate
  --abbrev-commit --all --pretty=oneline'
4 $ git lg
```



"Cherry-pick" sur Delicious Insights

81 / 85



"La remise" sur Delicious Insights



"Ignorer des fichiers" sur Delicious Insights

Quel casse tête tous ces HEAD !

A diagram showing three overlapping circles. The top-left circle is red and labeled "HEAD~2". The top-right circle is orange and labeled "HEAD~3". The bottom circle is blue and labeled "HEAD". The circles overlap in a central region.

Les différentes formes de HEAD expliquées sur StackOverflow

